

**SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES**  
**SIMATS ENGINEERING**  
**COMPUTER SCIENCE AND ENGINEERING**  
**CSA17 - ARTIFICIAL INTELLIGENCE**

**Experiment No. 11: Python Program to Implement Map Coloring Problem using CSP**

**Aim**

To write a Python program to solve the Map Coloring problem as a Constraint Satisfaction Problem (CSP), assigning a color to each region of a map such that no two adjacent (neighboring) regions share the same color, using the backtracking search technique.

**Algorithm**

1. Start.
2. Represent the map as a graph using a dictionary, where each key is a region and its value is the list of neighboring regions.
3. Define the list of available colors.
4. Create an empty dictionary 'assignment' to store the color assigned to each region.
5. Define a function is\_consistent(region, color, assignment) that returns False if any already-assigned neighbor of the region has the same color, and True otherwise.
6. Define a recursive function backtrack(assignment): a) If every region has been assigned a color, return the assignment. b) Select the next unassigned region. c) For each color in the color list, check is\_consistent(); if consistent, assign the color to the region and recursively call backtrack(). d) If the recursive call succeeds, return the result; otherwise remove the assignment and try the next color.
7. If no color can be consistently assigned to a region, return failure and backtrack to the previous region.
8. Call backtrack() with an empty assignment to begin solving the CSP.
9. Print the final color assigned to each region.
10. Stop.

**Pseudocode**

```
BEGIN
  INPUT: graph (adjacency list of regions), colors
  CREATE assignment = {}
  FUNCTION is_consistent(region, color, assignment)
    FOR each neighbor IN graph[region] DO
      IF neighbor IN assignment AND assignment[neighbor] == color THEN
        RETURN False
      END IF
    END FOR
    RETURN True
  END FUNCTION
  FUNCTION backtrack(assignment)
    IF length(assignment) == length(graph) THEN
      RETURN assignment
```

```

END IF
region = SELECT next unassigned region
FOR each color IN colors DO
    IF is_consistent(region, color, assignment) THEN
        assignment[region] = color
        result = backtrack(assignment)
        IF result != failure THEN
            RETURN result
        END IF
        REMOVE assignment[region]
    END IF
END FOR
RETURN failure
END FUNCTION
CALL backtrack(assignment)
PRINT assignment
END

```

## Result

The Python program to implement the Map Coloring problem using CSP was executed successfully. For a map with regions WA, NT, SA, Q, NSW, V and T, and 3 available colors (Red, Green, Blue), the backtracking search assigned colors such that no two adjacent regions received the same color, producing a valid solution: WA=Red, NT=Green, SA=Blue, Q=Red, NSW=Green, V=Red, T=Red. Thus, the CSP-based Map Coloring algorithm was implemented and verified successfully using Python.

## Sample Output:

```

$ python3 map_coloring.py
Regions and Neighbors:
WA  -> ['NT', 'SA']
NT  -> ['WA', 'SA', 'Q']
SA  -> ['WA', 'NT', 'Q', 'NSW', 'V']
Q   -> ['NT', 'SA', 'NSW']
NSW -> ['SA', 'Q', 'V']
V   -> ['SA', 'NSW', 'T']
T   -> ['V']

Available Colors: ['Red', 'Green', 'Blue']

Solving Map Coloring using Backtracking (CSP)...

Color Assignment:
WA  : Red
NT  : Green
SA  : Blue
Q   : Red
NSW : Green
V   : Red
T   : Green

Map Coloring completed successfully!

```